# Module 2: Cloud Platform Deep Dive (Infrastructure & Security Basics)



## Core Compute & IAM: Lab

# Compute

**Compute is the processing power you rent to run your software.** In the cloud, that usually means a virtual machine (VM): A computer simulated in software on shared hardware, a slice of a big physical server in a provider's data centre, made to behave like your own computer with its own CPU, RAM, disk, OS, and IP address.

You already know this from VirtualBox. The difference is that someone else owns and maintains the hardware, and you pay by the second or hour.

What you choose when you launch a VM(Anatomy of a VM):

- Region/zone: where it physically runs, i.e. geographic location (e.g. `europe-west1`), zone = data centre within it. Why it matters: latency, compliance, resilience.
- Machine type: how much CPU and RAM (small `e2-micro` for a test, bigger for a database) (could be General purpose / compute-optimized / memory-optimized)
- Image: the OS (Ubuntu, Debian, Windows)
- Boot disk: storage the OS lives on
- Network: which VPC/subnet, and whether it gets a public IP
- SSH key/access: how you log in

**Analogy:** Renting a flat. You pick the location (region), size (machine type), furnishing (image), and whether it has a street-facing door (public IP). The landlord handles the building; you handle what happens inside.

Lifecycle matters for cost:

- Running: you pay for CPU, RAM, and disk
- Stopped: you stop paying for CPU/RAM, but the disk still costs money
- Deleted: everything is gone and billing stops

Beyond VMs, compute includes containers (Cloud Run, ECS) and serverless functions (Week 3), but a VM is the foundation we will start with.

# Compute services

- Virtual machines (EC2 / Compute Engine): A full computer you manage: OS, software, patches. Most control, most work. Like renting an empty flat and furnishing it yourself.
- Containers (ECS, EKS / Cloud Run, GKE): Your app and its dependencies packaged together, running on shared infrastructure. Less to manage, faster to start. Like renting a furnished room.
- Serverless (Lambda / Cloud Functions): You upload code, the cloud runs it only when triggered. No servers to manage, billed per execution. Like a taxi: you don't own or maintain the car, you pay per ride.
- Managed platforms (App Engine / Elastic Beanstalk): You hand over your app and the platform handles servers and scaling.

# IAM (Identity and Access Management)

**IAM answers one question: who can do what on which resource?** Without it, anyone with your account login could delete everything. With it, you control exactly what each person or application is allowed to do. The three parts:

1. Principal (who): a user, a group, or a service account
2. Role/permission (what): the actions allowed, like "view files" or "delete VMs"
3. Resource (which): the thing being accessed, like one bucket or one project

Analogy: A hotel key card. Your card (identity) opens your room (resource) and the gym (permissions), but not other guests' rooms or the manager's office. The front desk can issue a card that opens the whole building, but giving everyone that is a security disaster.

**Principal role types:**

- User: a human with a login
- Group: a collection of users, so you grant access once instead of per person
- Service account: an identity for an application or VM, not a person. For example, a VM that needs to read from a bucket uses a service account instead of someone's personal credentials.

**Note: Principle of least privilege.** Give only what's needed, nothing more. Bad: Owner for a developer. Good: Storage Object Viewer on one bucket.

**Authentication vs authorization**

- Authentication: proving who you are (password, MFA)
- Authorization: what you're allowed to do once you're in (IAM)

The hotel analogy: showing ID at the desk versus which doors your card opens.

# Roles vs policies

A policy is a written rule that says who may do what on which resource. In AWS, a policy is a JSON document:

```
json

        {

          "Effect": "Allow",

          "Action": "s3:GetObject",

          "Resource": "arn:aws:s3:::my-bucket/*"

}
```

As a sentence: "Allow the action GetObject on everything inside my-bucket." Three parts: Effect (Allow/Deny), Action (what), Resource (where).

In GCP, you don't write the JSON. You bind a role to a principal on a resource: "Give `classmates-in-team` the `Storage Object Viewer` role on `my-bucket`." A role is a pre-built bundle of permissions. Basically, in GCP, you bind a role (a bundle of permissions) to a principal on a resource while on AWS, you attach a policy (a JSON document of allowed actions) to a user, group, or role. Same idea, different vocabulary.

**Least privilege (the key principle):** Give only the permissions needed for the job, nothing more.

- Bad: making a developer Owner of the whole project so they can read one bucket
- Good: giving them Storage Object Viewer on that one bucket

If their account is compromised, the damage is limited to what they could access.

# Core Compute & IAM together

A developer pushes an access key to GitHub. 5 minutes later, bots launch 200 servers mining crypto. The bill: $50,000." This is why we learn IAM and cost controls alongside compute, not after.

By the end of this topic you will... Launch a VM, configure IAM securely, manage storage and access, set cost alerts.

Check that you have an AWS/GCP account created and Billing enabled…

# Object vs block storage

Both store data, but they behave very differently.

**Block storage (EBS / Persistent Disk):** Raw disk space attached to a VM. The VM's operating system formats it and uses it like a normal hard drive. Fast, good for operating systems and databases. Usually tied to one VM at a time.

**Object storage (S3 / Cloud Storage):** Files stored as objects in buckets, accessed over the internet by URL or API. Not mounted like a drive. Cheap, practically unlimited, great for backups, images, videos, logs, and static websites. You can't edit part of a file; you replace the whole object.

Analogy: Block storage is the hard drive inside your laptop. Object storage is Google Drive.

| | Block | Object |
|---|---|---|
| Attached to | A VM | Nothing (accessed via API/URL) |
| Looks like | A Hard Drive | A folder on the internet |
| Best for | OS, Databases | Backups, media, logs |
| Speed | Fast | Slower |
| Cost | Higher per GB | Cheaper per GB |

# Launch a VM: Lab

Walk-through:

1. Console → Compute Engine → Create Instance
2. Name it (`my-vm`), pick a region and zone
3. Machine type: `e2-micro`
4. Boot disk: Ubuntu 22.04 LTS, 10 GB
5. Firewall: tick "Allow HTTP traffic"
6. Click Create, then wait for the green tick
7. Click SSH to open the browser terminal
8. Run `whoami`, `uname -a`, `sudo apt update && sudo apt install nginx -y`
9. Open the external IP in a browser and see the nginx welcome page

# Managing storage and access: Lab

1. Cloud Storage → Create bucket (globally unique name, same region as the VM, standard class)
2. Leave "Prevent public access" on
3. Upload a text file or image
4. Open the file's URL in an incognito window. It should be denied. Showing how "Private by default is the safe default."
5. Grant the test user Storage Object Viewer on this bucket. You can now read it.

# Compute pricing + free tier

Cloud pricing is how you decide how much commitment you're willing to make in return for a discount.

**On-demand:** Pay by the second or hour with no commitment. Start and stop any time. Most flexible, most expensive per hour. Best for testing, learning, and unpredictable workloads.

**Reserved / committed use:** Commit to 1 or 3 years of usage in exchange for a large discount (often 30-70%). Best for steady workloads that run constantly, like a production database. Like a gym membership: cheaper per visit if you actually go.

**Spot / preemptible:** Use the provider's spare capacity at up to ~90% off, but the provider can take it back with short notice. Best for interruptible work like batch jobs and data processing. Never use for anything that must stay up. Like a standby airline ticket: cheap, but you might get bumped.

**Free tier:** Providers give limited free usage so you can learn. It comes in three forms: always-free (small monthly allowances), 12-month free (for new accounts), and trial credits. Free tier has limits: the wrong machine type, region, or extra services like a static IP or NAT gateway will cost money. Check the provider's current free tier terms, since they change.

# Setting Cost Alerts: Lab

Walk-through:

1. Billing → Budgets & alerts → Create budget
2. Scope: project
3. Amount: a small figure (e.g. $5 or ₦ equivalent)
4. Thresholds: 50%, 90%, 100%
5. Notifications: email
6. Save, then open the Billing reports page and see where costs appear by service

Note: At the end of every lab or practice you do, check the billing page and the resources list. Alerts do not stop resources. They only notify. (Automated shutdown is possible but needs extra setup, so mention it as a future topic.) Billing data can lag by hours, so a small overspend can slip through before the alert.

Common cost traps. Forgotten VMs, unattached disks, static IPs, NAT gateways, oversized instances.

# Review and Practice Questions

- What's the difference between a region and a zone?
- Why use a service account?
- Does a stopped VM still cost you? What do you pay for?
- What does least privilege mean?

Next week: How do we network and protect these resources? Welcome to Networking & Security

Extra learning materials

https://www.gnu.org/software/bash/manual/bash.html

https://docs.cloud.google.com/iam/docs/overview

https://unidel.edu.ng/focelibrary/books/A-Complete-Guide-to-the-Google-Cloud-Platform.pdf

https://www.skills.google/focuses/2794?parent=catalog&path=22